1) Diseñe un interpretador para la siguiente gramática que realiza operaciones con notación infija:

Valores denotados: Texto + Número + Booleano + ProcVal

Valores expresado: Texto + Número + Booleano + ProcVal

<programa> := <expresion>

un-programa (exp)

<expresion> := <numero>

numero-lit (num)

:= "\""<texto> "\""

texto-lit (txt)

:= <identificador>

var-exp (id)

:= (<expresion> <primitiva-binaria> <expresion>)

primapp-bin-exp (exp1 prim-binaria exp2)

:= <primitiva-unaria> (<expresion>)

primapp-un-exp (prim-unaria exp)

<primitiva-binaria> := + (primitiva-suma)

:= ~ (primitiva-resta) Observe que al tratarse de notación infija, resta la trabajaremos con la virgulilla para evitar confusión entre números negativos y la operación resta. Ejemplo: (4 ~ 5) -> -1 y (4 ~ -5) ->9

:= / (primitiva-div)

:= * (primitiva-multi)

:= concat (primitiva-concat)

:= > (primitiva-mayor)

:= < (primitiva-menor)

:= >= (primitiva-mayor-igual)

:= <= (primitiva-menor-igual)

:= != (primitiva-diferente)

:= == (primitiva-comparador-igual)

<primitiva-unaria>:= longitud (primitiva-longitud)

:= add1 (primitiva-add1)

:= sub1 (primitiva-sub1)

:= neg (primitiva-negacion-booleana)

Tenga en cuenta que:

<numero>: Debe definirse para valores decimales y enteros (positivos y negativos)

<texto>: Debe definirse para cualquier texto escrito en racket. Un texto en este lenguaje son palabras que inician con una o más letras, seguidos de posibles números o el guión bajo para separar palabras "_". Defina el texto en la especificación léxica sin las comillas y posteriormente escape las comillas en la gramática como se muestra arriba.

<identificador>: En este lenguaje todo identificador iniciará con el símbolo @, es decir las variables @x y @z son válidas

2)Defina un ambiente inicial con las variables (@a @b @c @d @e) con valores (1 2 3 "hola" "FLP") y modifique su función evaluar-expresión para que acepte dicho ambiente.

-Diseñe una función llamada (buscar-variable) que recibe un símbolo (identificador) y un ambiente, retorna el valor si encuentra la variable en el ambiente. En caso contrario: "Error, la variable no existe"

Pruebe:

--> @a

1

--> @b

2

--> @e

"FLP"

3) Implemente los Booleanos:

En una expresión numérica, 0 es falso, cualquier otro caso es verdadero. Para esto diseñe la función valor-verdad? que realiza esta verificación.

--> (5 > 9)

0

--> (6 <= 6)

1

4) Extienda la gramática con condicionales:

<expresion> := Si <expresion> "{" <expresion> "}" "sino" "{" <expresion> "}"

condicional-exp (test-exp true-exp false-exp)

Debe probar:

```
--> Si (2+3) {
          2
      }sino {
         3
      }
2
--> Si (longitud(@d) ~ 4) { 2 } sino { 3 }
3
```

5) Implemente declaración de variables locales:

<expresion> := declarar ({<identificador> = <expresion> ';' }*)) { <expresion> }

variableLocal-exp (ids exps cuerpo)

Debe probar:

```
--> declarar (@x=2;@y=3;@a=7;){
      (@a+(@x~@y))
   }
6
--> declarar (@x=2;@y=3;@a=7;) {
     (@a+@b)
  }
9
```

6) Extienda la gramática para crear procedimientos

<expresion> := procedimiento (<identificador>*(',')) "{" <expresion> "}"

procedimiento-ex (ids cuero)

Para esto debe definir un datatype para la cerradura (o ProcVal) que debe tener 3 campos:

1. Lista ID del procedimiento

2. Cuerpo del procedimiento

3. Ambiente donde fue declarado

```
(define-datatype procVal procVal?
  (cerradura
   (lista-ID (list-of symbol?))
   (exp expresion?)
   (amb ambiente?)
   )
  )
```

Debe probar:

```
--> procedimiento (@x,@y,@z) {
          ((@x+@y)+@z)
      }
```

#(struct:cerradura (@x @y @z) #(struct:primapp-bin-exp #(struct:primapp-bin-exp #(struct:var-exp @x) #(struct:primitiva-sum) #(struct:var-exp @y)) #(struct:primitiva-sum) #(struct:var-exp @z)) #(struct:extendido (@a @b @c @d @e) (1 2 3 "hola" "FLP") #(struct:vacio)))

Se debe retornar una cerradura

7) Extienda la gramática para evaluar procedimientos:

<expresion> := "evaluar" expresion (expresion *(",")) finEval

app-exp(exp exps)

Debe probar:

```
-->  declarar (
```

```
      @x=2;
      @y=3;
      @a=procedimiento (@x,@y,@z) { ((@x+@y)+@z) } ;
     ) {
        evaluar @a (1,2,@x) finEval
      }
5


--> declarar (
     @x=procedimiento (@a,@b) { ((@a*@a) + (@b*@b)) };
     @y=procedimiento (@x,@y) { (@x+@y) } ;
    ) {
      ( evaluar @x(1,2) finEval + evaluar @y(2,3) finEval )
     }
10
--> declarar (
      @x= Si (@a*@b) { (@d concat @e) } sino { longitud((@d concat @e)) };
      @y=procedimiento (@x,@y) { (@x+@y) } ;
   ) {
      ( longitud(@x) * evaluar @y(2,3) finEval )
     }
35
```

8) Extienda la gramática para incluir llamados recursivos. Proponga una definición en la gramática e impleméntela.

9) Utilización del lenguaje de programación:

Sólo esta parte del taller será evaluada:

a) **10pts.** Escriba un programa en su lenguaje de programación que contenga un procedimiento **@sumarDigitos** que le permita sumar los dígitos de un número entero positivo. **evaluar @sumarDigitos(147) finEval** deberá retornar 12

b) **5pts.** Escriba un programa en su lenguaje de programación que contenga un procedimiento recursivo **@sumaNaturales(n)** que calcule la suma de los primeros (n) enteros positivos. Como la gramática para funciones recursivas debe ser propuesta por el grupo, incluya dos ejemplos de uso para (n=5), el resultado debe ser 15; para (n=10), el resultado debe ser 55.

c) **10pts.** Escriba un programa en su lenguaje de programación que contenga un procedimiento recursivo **@multiplicar(a,b)** que calcule el producto de dos enteros no negativos mediante sumas sucesivas, sin utilizar directamente la primitiva de multiplicación para calcular el resultado. Si no se evidencia el uso de recursión, el ejercicio no será valido. Incluya un llamado a la función recursiva:

"evaluar @multiplicar(4, 3) finEval"

Deberá retornar 12.

d) **15pts.** Escriba un programa que sume los números pares en un rango de valores positivos [a,b], donde siempre se cumple en la invocación a < b: Por ejemplo "evaluar @sumaParesRango (2, 7) finEval ", Deberá retornar 12, correspondiente a (2+4+6).

e) **25pts.** En python se puede utilizar algo que se llaman decoradores (por favor leer aquí). Crea una función @integrantes que muestre los nombres de los integrantes del grupo y adicionalmente crea un decorador que al invocarlo salude a los integrantes:

. . .

@integrantes = //un procedimiento que retorna un string con los nombres de los integrantes, ejemplo"Robinson_y_Sara"

. . .

@saludar = //un procedimiento que recibe un procedimiento y retorna otro procedimiento que le agrega el string "Hola:" a la invocación del procedimiento que recibió en sus argumentos.

. . .

// Creación del decorador

@decorate = evaluar @saludar (@integrantes) finEval

. . .

// Invocación del decorador

evaluar @decorate ( ) finEval //Deberá retornar "Hola:Robinson_y_Sara"

Esto quiere decir que la función @saludar recibe como parámetro la función @integrantes y a su vez, retorna una función que se almacena en la variable @decorate. Esta última función, al ser invocada, deberá retornar un saludo a los integrantes (cualquier implementación sin el concepto de decorador no será evaluada).

f) **35pts.** Modifique el ejercicio anterior para que el decorador reciba como parámetro otro mensaje que debe ponerse al final de todo el string (cualquier implementación sin el concepto de decorador no será evaluada). Ejemplo

// Invocación del decorador

evaluar @decorate ("_ProfesoresFLP") finEval //Deberá retornar "Hola:Robinson_y_Sara_ProfesoresFLP"

Aclaraciones

1. El taller deberá desarrollarse en los grupos de trabajo.
2. La solución del taller (**interpretador.rkt**) debe ser subida al campus virtual. Sin embargo, un integrante de cada grupo deberá crear un repositorio privado en GitHub y cada integrante del grupo deberá aportar al desarrollo del trabajo (esto se deberá evidenciar en los commits). Deberán agregar al repositorio al usuario **robinsonduque**
3. En las primeras líneas del interpretador deben estar comentados los nombres y los códigos de los estudiantes participantes junto con la URL del repositorio en GitHub
4. Se debe incluir para cada procedimiento de la implementación del interpretador un comentario que explique lo que realiza y para qué es empleada
5. En el taller hay ejercicios (a,b,c,d,e,f) en los cuales deben escribir funciones en el lenguaje dado. Éstos códigos deben ser entregados al final del archivo de racket de forma tal que queden comentados y con la respectiva numeración.
6. En caso de duda sobre la participación de algún integrante dada su participación en el desarrollo del taller (acorde al repositorio), podrán ser llamados a sustentación individual. En la sustentación obtendrán una Factor con valor entre 0 y 1 que se multiplicará por la nota obtenida por el grupo. Este ajuste sólo afectará al estudiante que se cite a sustentación salvo que se cite al grupo en pleno.

Entregas Tardías o por Otros Medios

1. Este taller sólo se recibirá a través del campus virtual. Adicionalmente, sólo se

evaluarán el documento solicitado (**interpretador.rkt**). Cualquier otro tipo de correo o nota aclaratoria será descartado.

2. Las entregas tarde serán penalizadas así: (-1pt) por cada hora de retraso o fracción. Por ejemplo, si usted realiza su entrega y el campus registra las 12:00 am (i.e., 1 seg después de la hora de entrega), usted está incurriendo en la primer hora de retraso. Asegurese con mínimo dos horas de anticipación que el link de carga funciona correctamente toda vez que es posible incurrir en una entrega tardía debido a los tiempos de respuesta.